

Data Analysis with Python

Week 2: Programming Fundamentals

Justinas Grigaitis, 2025-09-25

Plan for today

Programming Fundamentals

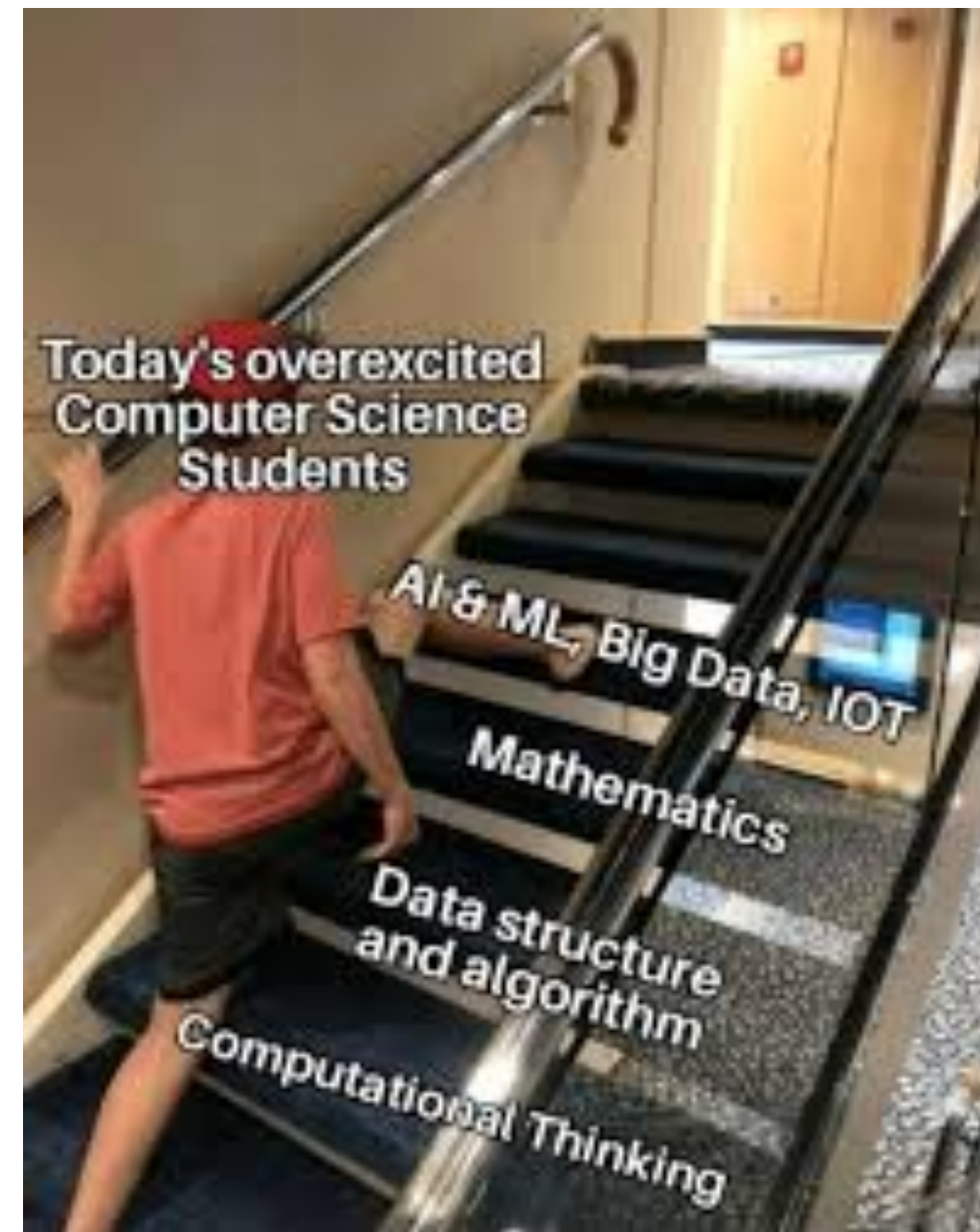
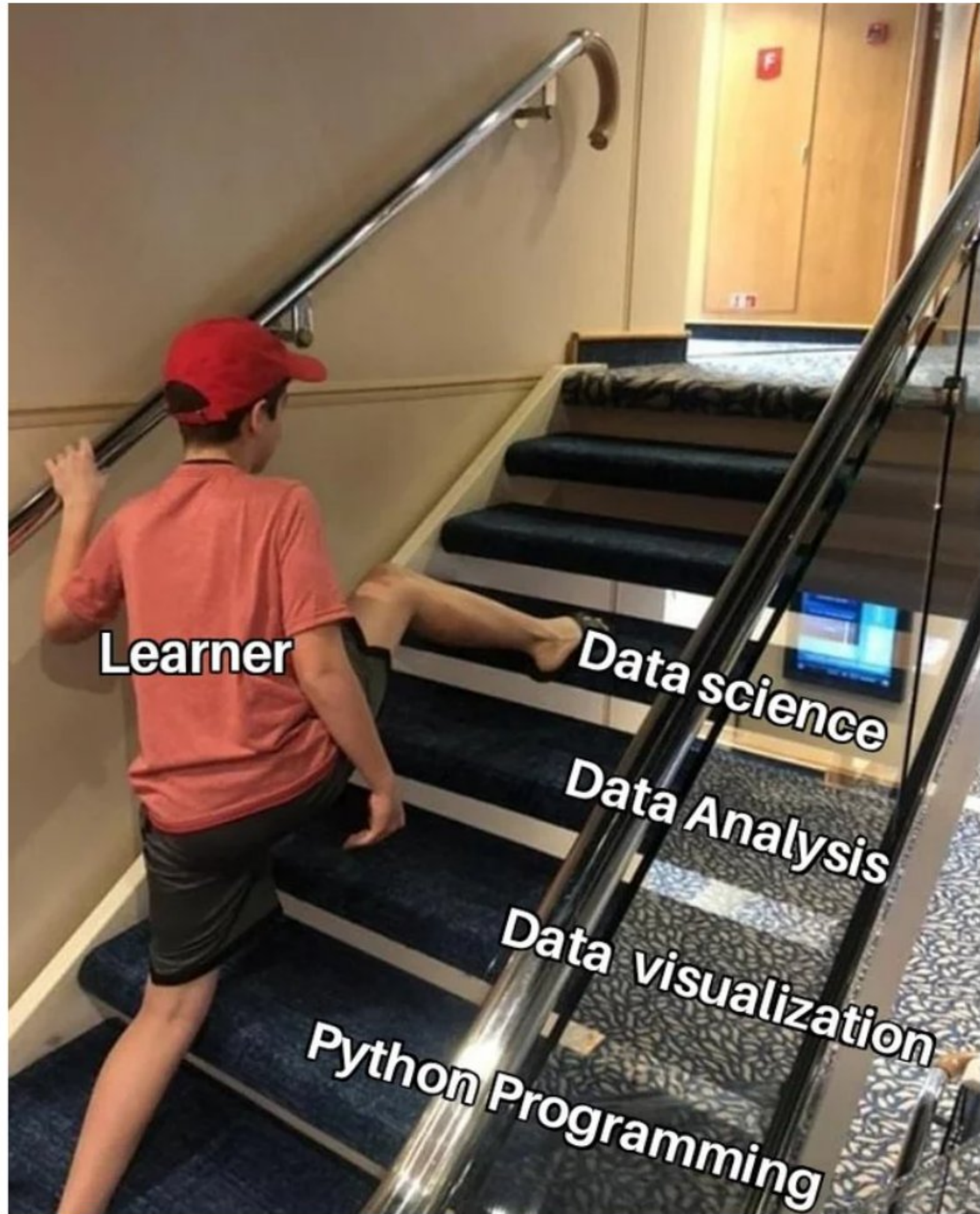
- Intro: feedback
- First half: programming fundamentals - PPT + code
- 15 min break - fix issues
- Second half: GitHub and Streamlit playground

Feedback

I'm moved - you're the best!

- Excellent work from all of you
- Programming is inherently self-paced and self-motivated activity
- View these weekly sessions as check-ins:
 - Motivation and accountability = consistency
 - Recap of most important topics = guided focus
 - Feedback and help with the hardest parts = support mechanism
 - Peer group and my tips from experience = in-person value

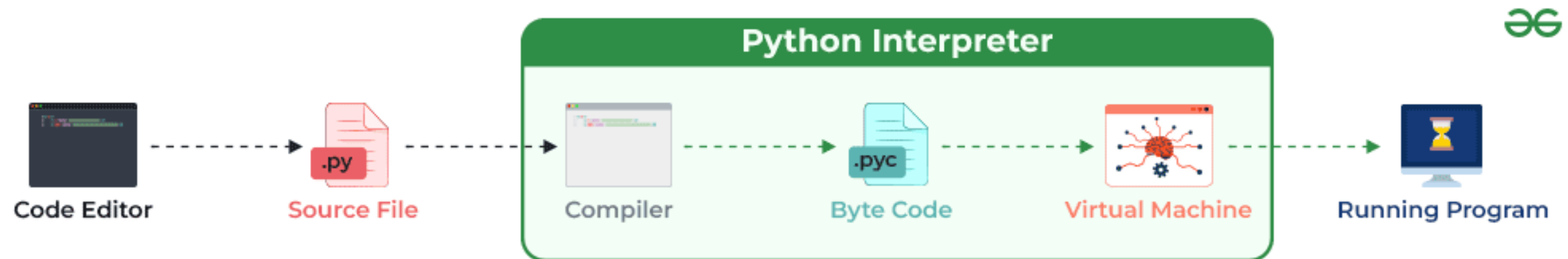
Programming Fundamentals



Human and machine code

Text → Interpreter → Machine code/execution → Output

- Source code is just text (in most languages!)
- Language-specific interpreter → text into machine code
- Machine executes instructions → user sees output
- Practice: Write your first website; run Python code/file in terminal



What's an IDE?

Integrated Development Environment

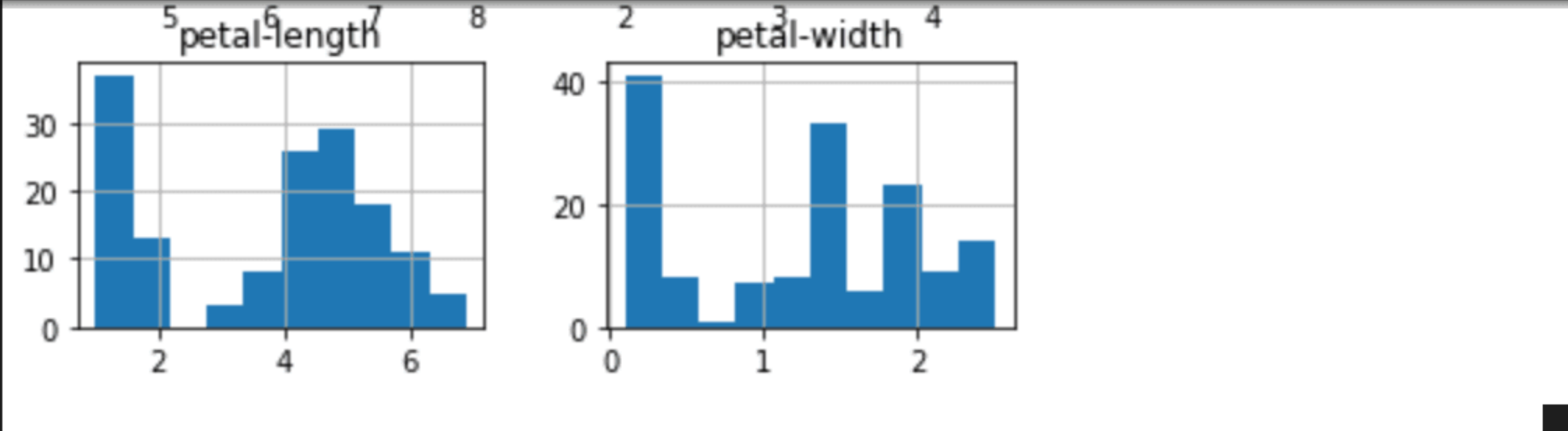
- VS Code (general), R Studio (for R), PyCharm/Spyder (for Python)
- Why use an IDE? - same as why use “MS Word” instead of Notepad
 - Code, terminal, and output in one place
 - Language-specific UX - code completion, help info, code formatting
 - File management, Git integration
 - AI copilots - don't use for now!

MLTutorial.py

```
45 plt.show()
46
47 Run Cell | Run Above | Debug Cell | Go to [8]
48 #%%
49 # scatter plot matrix
50 scatter_matrix(dataset)
51 plt.show()
52
53 Run Cell | Run Above | Debug Cell | Go to [9]
54 #%%# Split-out validation dataset
55 array = dataset.values
56 X = array[:,0:4]
57 Y = array[:,4]
58 validation_size = 0.20
59 seed = 7
60 X_train, X_validation, Y_train, Y_validation = model_selection
61
62 Run Cell | Run Above | Debug Cell
63 #%%
64 # Test options and evaluation metric
65 seed = 7
```

Interactive-1

Python 3.9.6 64-bit



scatter plot matrix...



Type code here and press Shift+Enter to run

PROBLEMS

OUTPUT

TERMINAL

JUPYTER: VARIABLES

DEBUG CONSOLE

Name	Type	Size	Value
array	ndarray	(150, 5)	[[5.1 3.5 1.4 0.2 'Iris-setosa'] [4.9 3.0 1.4 0.2 'Iris
dataset	DataFrame	(150, 5)	sepal-length sepal-width petal-length petal-w
names	list	5	['sepal-length', 'sepal-width', 'petal-length', 'pe
seed	int	7	7
url	str	68	https://raw.githubusercontent.com/jbrownlee/
validation_size	float	0.2	0.2
X	ndarray	(150, 4)	[[5.1 3.5 1.4 0.2] [4.9 3.0 1.4 0.2] [4.7 3.2 1.3 0.2

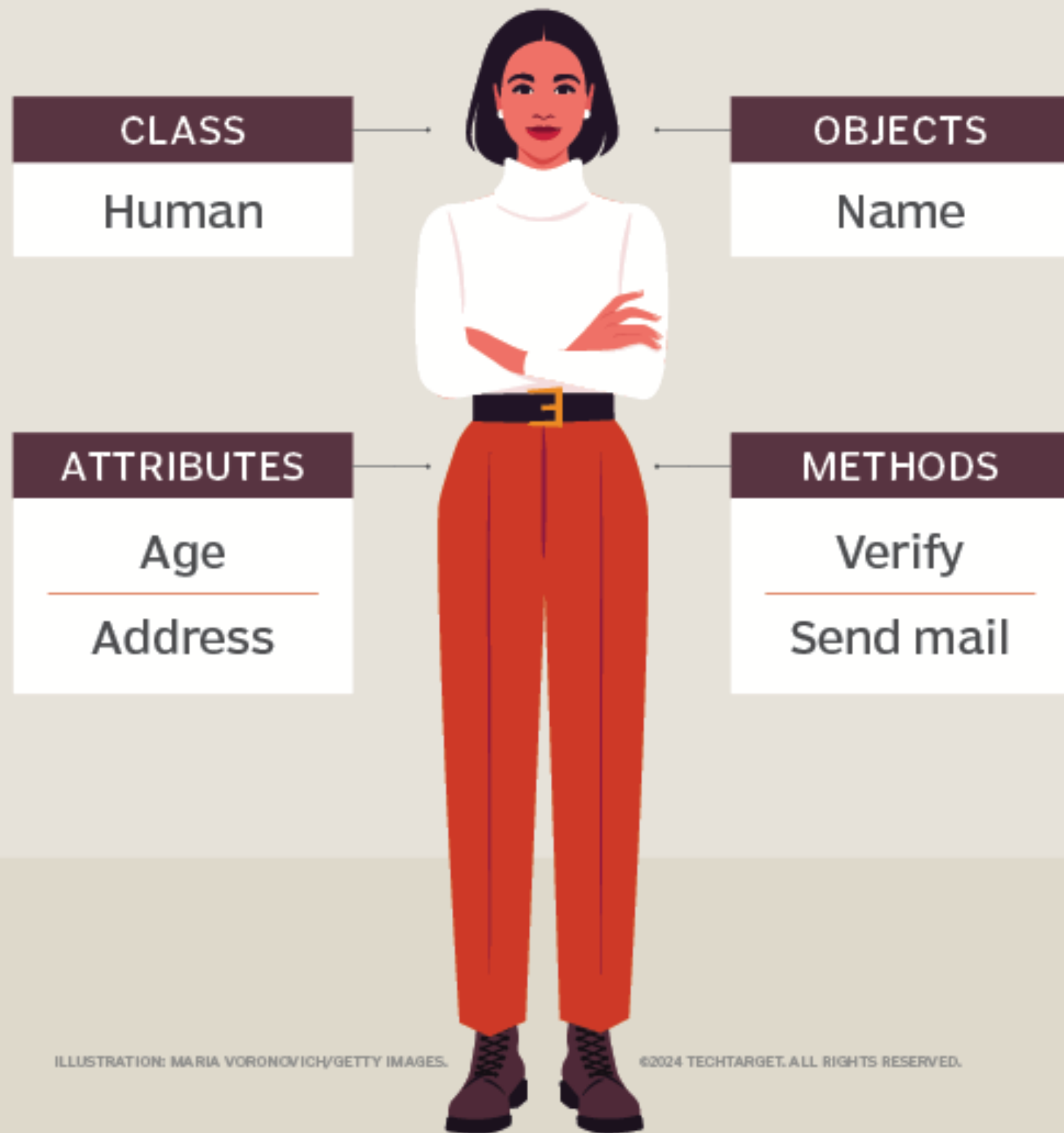
master Python 3.9.6 64-bit 0 0 Ln 53, Col 23 Tab Size: 4 UTF-8 CRLF Python

Object-oriented programming

Everything is an object

- Object - the core unit of any OOP language
 - Even *functions* are objects!
- Each object has a class (or type)
- Each class has its own functions, methods, and attributes
- You create objects (class instances) by assigning them to a variable name
- You can create new instances or modify objects in place (if mutable)

Object-oriented programming



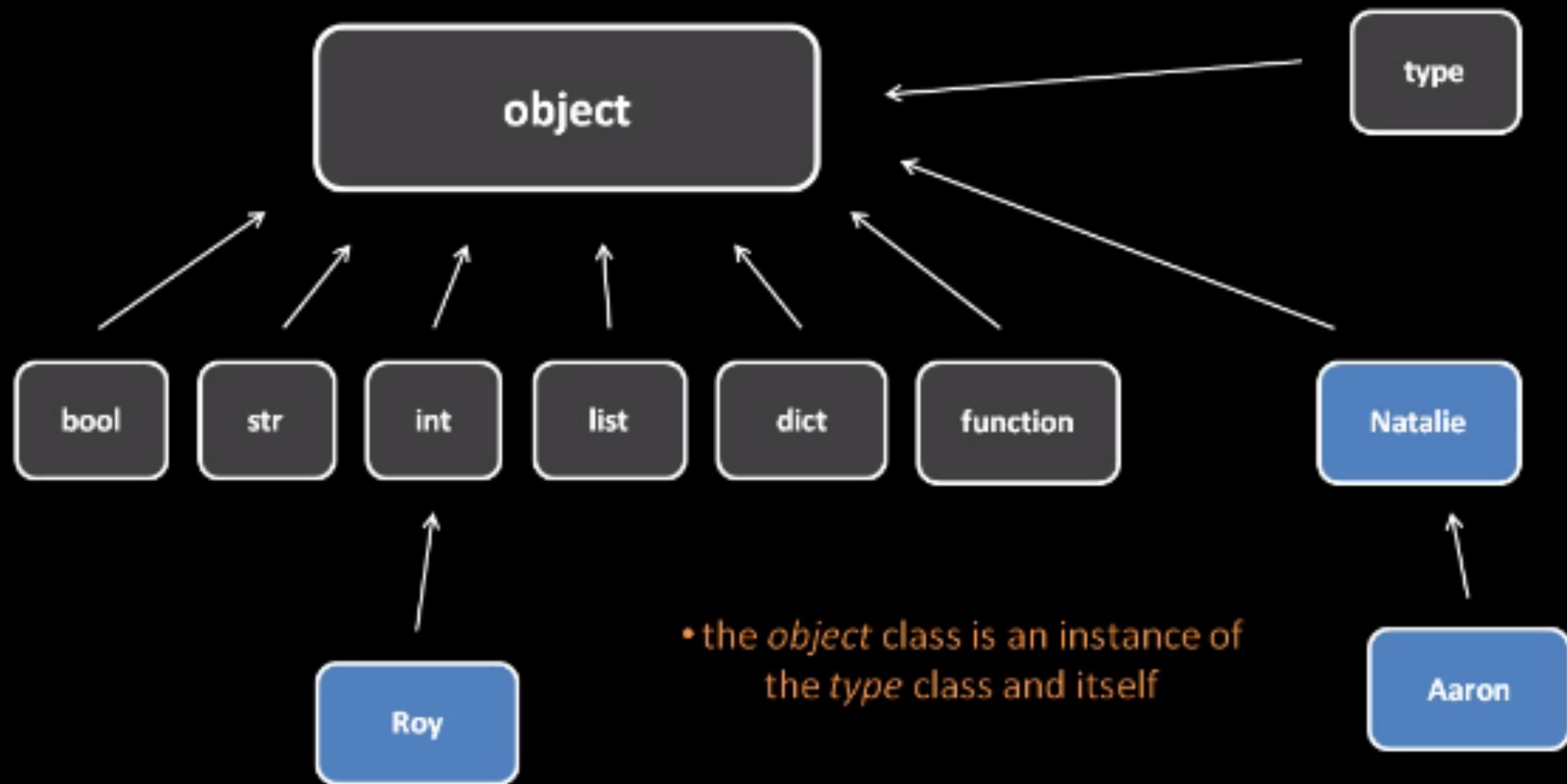
Python Class

```
1 class Car:
2     def __init__(self, year, make, model):
3         self.year = year
4         self.make = make
5         self.model = model
6
7     def startEngine(self, str = 'Vvvrroooooommmmm!!!'):
8         print(str)
9
10    def showYear(self):
11        return self.year
12
13 maserati = Car(2019, 'Maserati', 'Levante')
14 maserati # <__main__.Car object at 0x106645278>
15
16 maserati.model # 'Levante'
17
18 maserati.startEngine() # Vvvrroooooommmmm!!!
19 maserati.startEngine('sputter sputter misfire')
20 # sputter sputter misfire
21
22 maserati.showYear() # 2019
```

Main object types

From int to dataframe

- Base Python
 - Int, float (Numbers); string (Characters)
 - Boolean (True/False), None
 - List, tuple (Ordered collections)
 - Set, dictionary (Unordered collections)
- Third-party libraries (packages, modules)
 - Numpy - `np.array()` (core of data analysis)
 - Pandas - `pd.Series()`, `pd.DataFrame()` (labeled arrays)



- the *object* class is an instance of the *type* class and itself

- the *type* class is an instance of the *object* class and itself

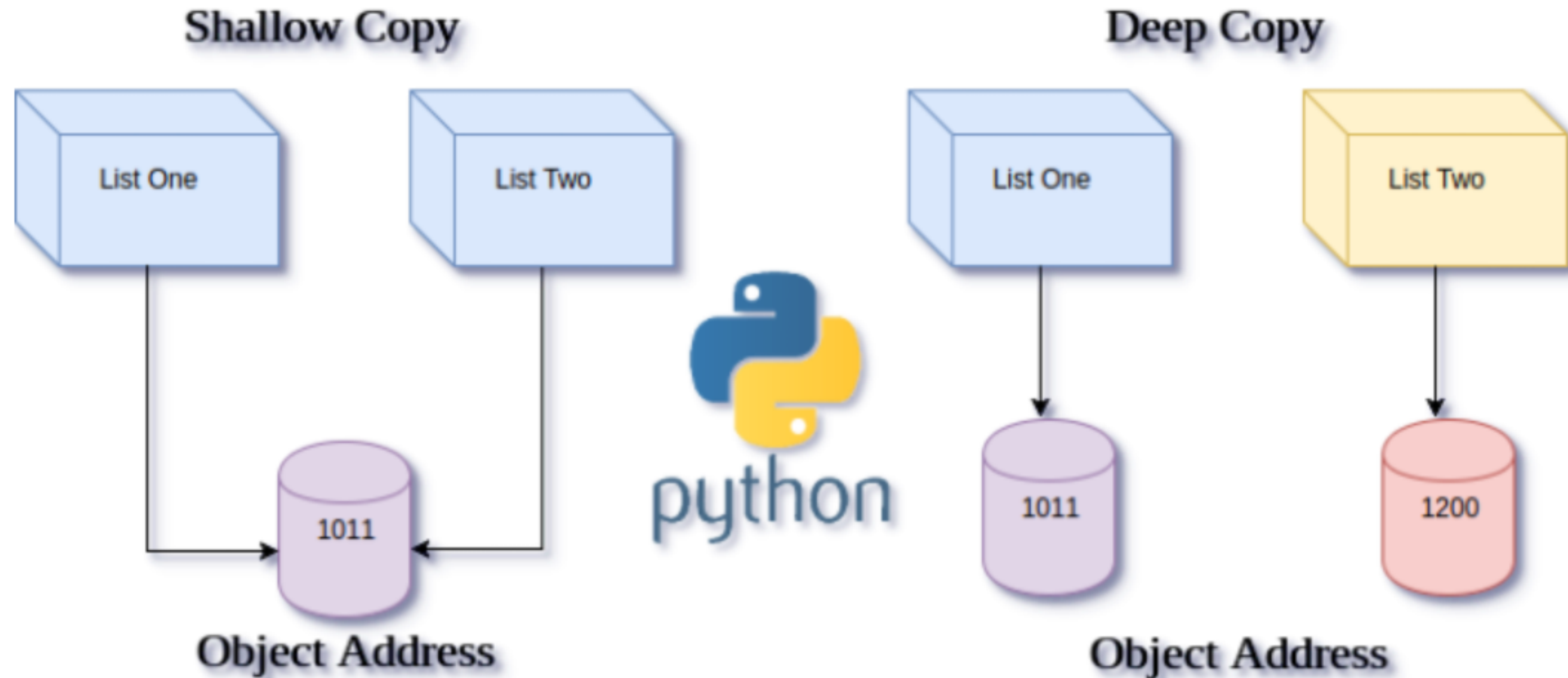
Building blocks of Python

And many other high level OOP languages

- Objects - new vs in-place, type casting
- Functions - built-in, third-party, user-defined
- Conditionals - comparison, flow control
- Loops - *for* and *while*
- Errors - debugging and handling
- File/user Input/Output (I/O) - back-end vs front-end

One step deeper

Copies vs References - Mutability



Collections of objects

Order, indexing, and mutability

Feature	Ordered Data Structures	Unordered Data Structures
Order of Elements	Elements are stored and retrieved in the order of insertion.	No guaranteed order of elements.
Indexing	Allows indexing (e.g., <code>arr[0]</code> for lists).	Does not allow direct indexing or slicing.
Mutability	Can be mutable (e.g., lists) or immutable (e.g., tuples).	Mutable (e.g., sets) or immutable (e.g., frozensets).
Duplicates	Allows duplicates (e.g., lists).	Does not allow duplicates (e.g., sets).
Use Cases	When the order of elements matters (e.g., sequencing, tasks).	When uniqueness is more important than order (e.g., fast membership checking).
Examples	Lists, Tuples, Strings, <code>OrderedDict</code> .	Sets, frozensets, Dictionaries (prior to Python 3.7).

Truthiness and Conditionals

Boolean, truth, and flow control

- Boolean takes one of the two “truth” values: True or False
- Comparison operators: ==, !=, <, <=, >, >=
- Logical operators: not, and, or + (parentheses)
- We mostly use Booleans in flow-control statements
 - If, elif, else
- Each object has a truth value!

Loops

Make the world go round

- Loops run a piece of code multiple times (fixed number or dynamic)
- “for loops” is our main tool
 - “while loops” are used a lot in software eng but less in data analysis
- “for loops” iterate over any collection (range, list, tuple, dictionary, dataframe)
- List and dictionary comprehension - one-liner for simple iteration
- Key functions: range(), enumerate(), zip()

Functions

A chunk of commands for repeated use

- Function is a **pre-defined** group of commands executed with a **function call**
- Functions take in **arguments**
 - Default, keyword, positional, arbitrary (args* and kwargs*)
 - These become **local** variables
- Functions can access **local** and **global** variables
- Functions may or may not **return** an object

Functions

Come in a few different flavours

- Functions vs methods
 - `sorted(my_list)`, `my_list.sort()`
- Built-in functions vs third-party libraries
 - `math (x+y)`, `print("hello world")`, `len("University of Zurich")`
 - Pandas, numpy, matplotlib (**`np.add()`**, **`pd.groupby()`**, **`plt.show()`**)
- User-defined functions
 - `def clean_data(df= my_data):`

User-defined functions

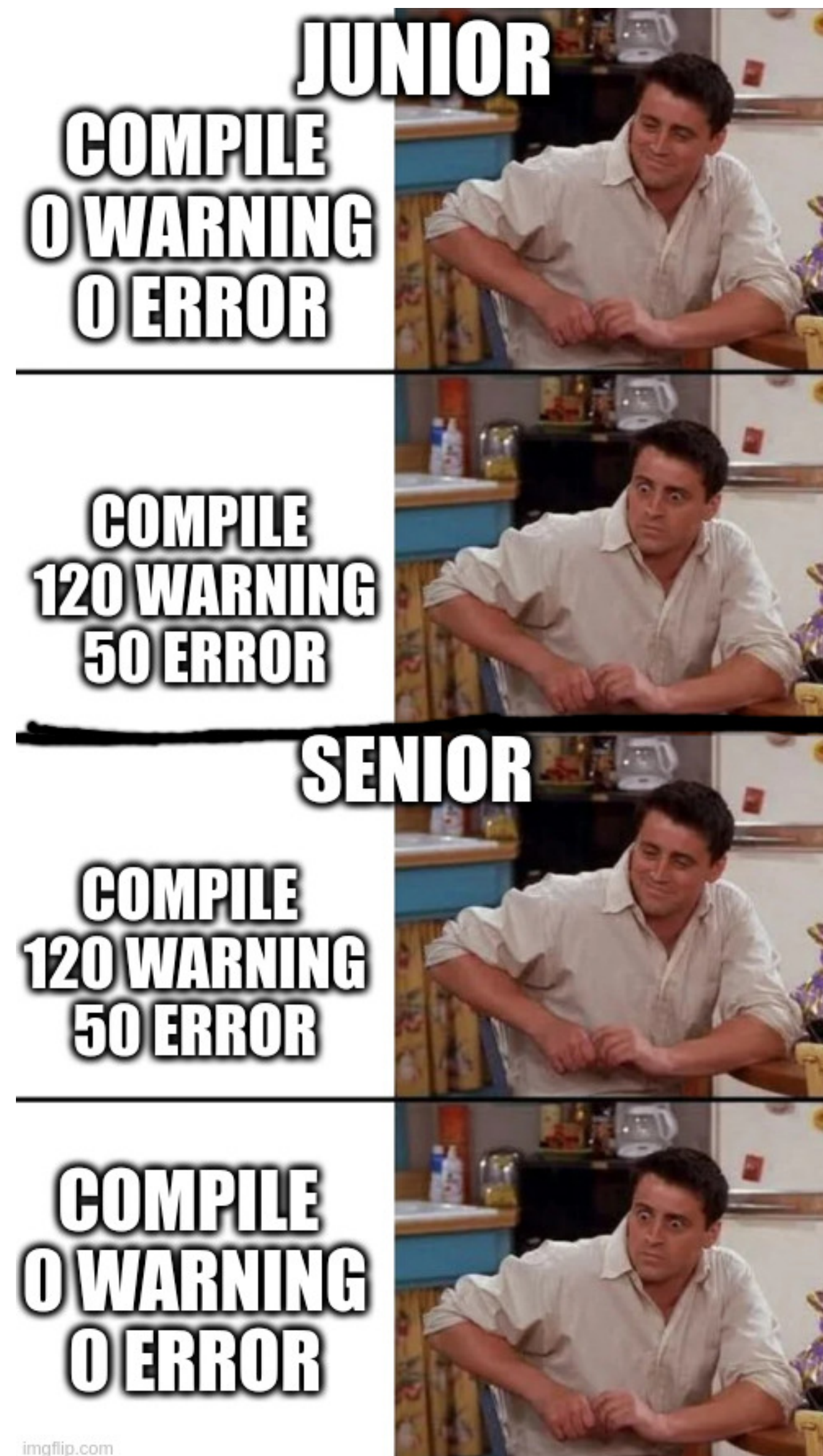
DRY! (do not repeat yourself)

- If you repeat the same 4-5 lines of code - turn it into a function
 - Give a short informative name
 - Add a brief “doc string” to explain what it is doing
- Use functions to structure your code
 - Even if code is not repeated - turn chunks of code into functions
 - The code is much cleaner to read, edit, and debug
- Use return statements and avoid modifying global variables in place

Debugging and error handling

Test of patience and perseverance

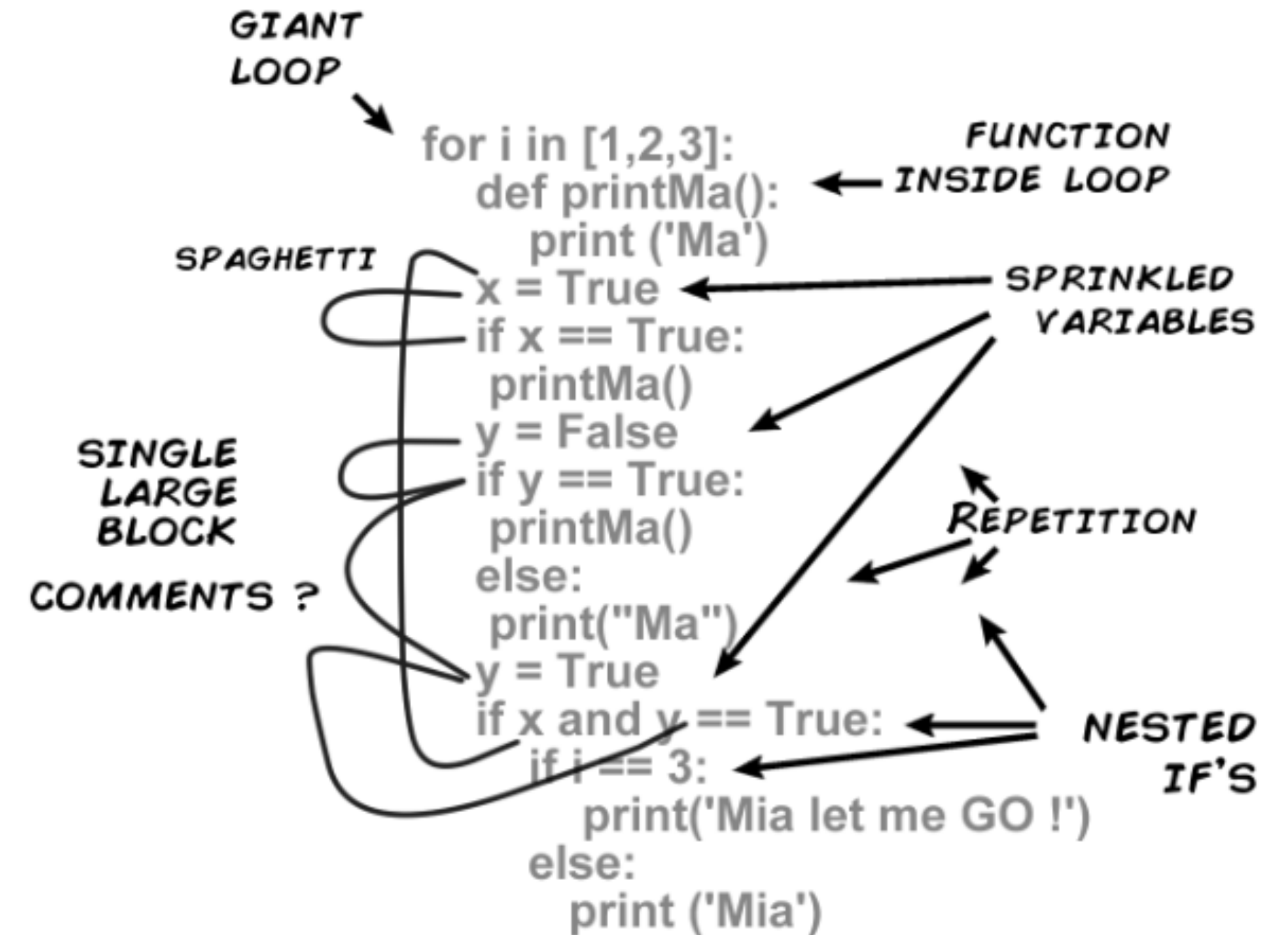
- Errors - tell you where and why you're wrong - computer is always right
- They are informative - find exact place where it broke and type of error
- Error handling - try: except: if you want to condition on error happening
- Debugging - use A LOT of print() statements for debugging
- Debugger in IDE
 - Helpful for complex functions and processes that take long
 - I rarely use it in data analysis



Code in style

Spaghetti code and official guidelines

- PEP 8 - official style guide
- Pseudo-code in advance, and keep track
- Small building blocks
- No spaghetti code
- Crucial with AI!!!
- Bonus: use linter and formatter in VS Code



PEP 20 - The Zen of Python

The Zen of Python

```
Beautiful is better than ugly.  
Explicit is better than implicit.  
Simple is better than complex.  
Complex is better than complicated.  
Flat is better than nested.  
Sparse is better than dense.  
Readability counts.  
Special cases aren't special enough to break the rules.  
Although practicality beats purity.  
Errors should never pass silently.  
Unless explicitly silenced.  
In the face of ambiguity, refuse the temptation to guess.  
There should be one-- and preferably only one --obvious way to do it.  
Although that way may not be obvious at first unless you're Dutch.  
Now is better than never.  
Although never is often better than *right* now.  
If the implementation is hard to explain, it's a bad idea.  
If the implementation is easy to explain, it may be a good idea.  
Namespaces are one honking great idea -- let's do more of those!
```

15 min break

Let's fix things

Playground

[**https://github.com/justgri/ba-programming-playground**](https://github.com/justgri/ba-programming-playground)

Python for front-end

Easy way to the outside world

- Python is primarily a back-end oriented language
- New libraries adapted it for front-end: Django, Flask, FastAPI, Plotly Dash
- More recently, a wave of very user-friendly tools: Streamlit, Gradio, Taipy, etc.
- Most importantly, a change in mindset in the age of GenAI
- Tutorials, videos: [fast prototyping](#), [data reports vs apps](#), <https://30days-tmp.streamlit.app/>

```
import streamlit as st
import pandas as pd
import numpy as np

st.write("Streamlit supports a wide range of data visualizations, i

all_users = ["Alice", "Bob", "Charly"]
with st.container(border=True):
    users = st.multiselect("Users", all_users, default=all_users)
    rolling_average = st.toggle("Rolling average")

np.random.seed(42)
data = pd.DataFrame(np.random.randn(20, len(users)), columns=users)
if rolling_average:
    data = data.rolling(7).mean().dropna()

tab1, tab2 = st.tabs(["Chart", "Dataframe"])
tab1.line_chart(data, height=250)
tab2.dataframe(data, height=250, use_container_width=True)
```

Share

Streamlit supports a wide range of data visualizations, including [Plotly](#), [Altair](#), and [Bokeh charts](#). 🇮🇹 And with over 20 input widgets, you can easily make your data interactive!

Users

Alice ×

Bob ×

Charly ×

× ▼

☐ Rolling average

Chart Dataframe



GitHub and Playground

Let's make it real right away

- git clone, git init
- git branch
 - git switch -c "student/initials"
- git pull, add, commit, push
 - **Always pull before pushing**
- <https://github.com/justgri/ba-programming-playground>

Next Steps

Next week - Intro to Data Analysis

Numpy, Pandas, and Data Cleaning

- Numpy arrays - vectorized operations
- Pandas Series and DataFrames - “Excel” in Python
- Data cleaning
 - Read csv and Excel files - one by one and with loops
 - Add, remove, rename columns, convert types
 - Edit and create columns with math and string operations
 - Filter data, remove duplicates, remove NAs

Homework (~5 hours)

Mastering the fundamentals

- Complete the Installation Guide
- Some more theory and practice topics (remaining from installation):
 - YouTube ([link](#)) - same guy as last week, but 2hr video with more details
 - Conditionals, Loops, and Functions: Ch. 6-8 in the [textbook](#)
- If you are really speedy:
 - Read intro to pandas tutorial: [link](#)
 - Read about streamlit ([link](#)) and make updates to your playground student page